

CrediSense AI

Rapport technique complet : architecture, pipeline de A à Z, changements, tests, speech

Projet : analyse automatisée de dossiers de crédit à la consommation (contexte bancaire tunisien)

Branche : `deploy/azure` · Dernier commit : `b8302511` · Rapport du 7 octobre 2026

1. En bref

CrediSense AI reçoit les pièces d'un client (CIN, fiche de paie, relevé bancaire, attestation de travail...), les lit par OCR, en extrait les données, puis calcule si le crédit demandé est supportable. Le client dépose sa demande sur un portail. Un agent de la banque vérifie les documents, lance l'analyse et obtient un résultat expliqué : décision, contrôles réglementaires, capacité d'emprunt, simulations de durée, points forts et points de vigilance.

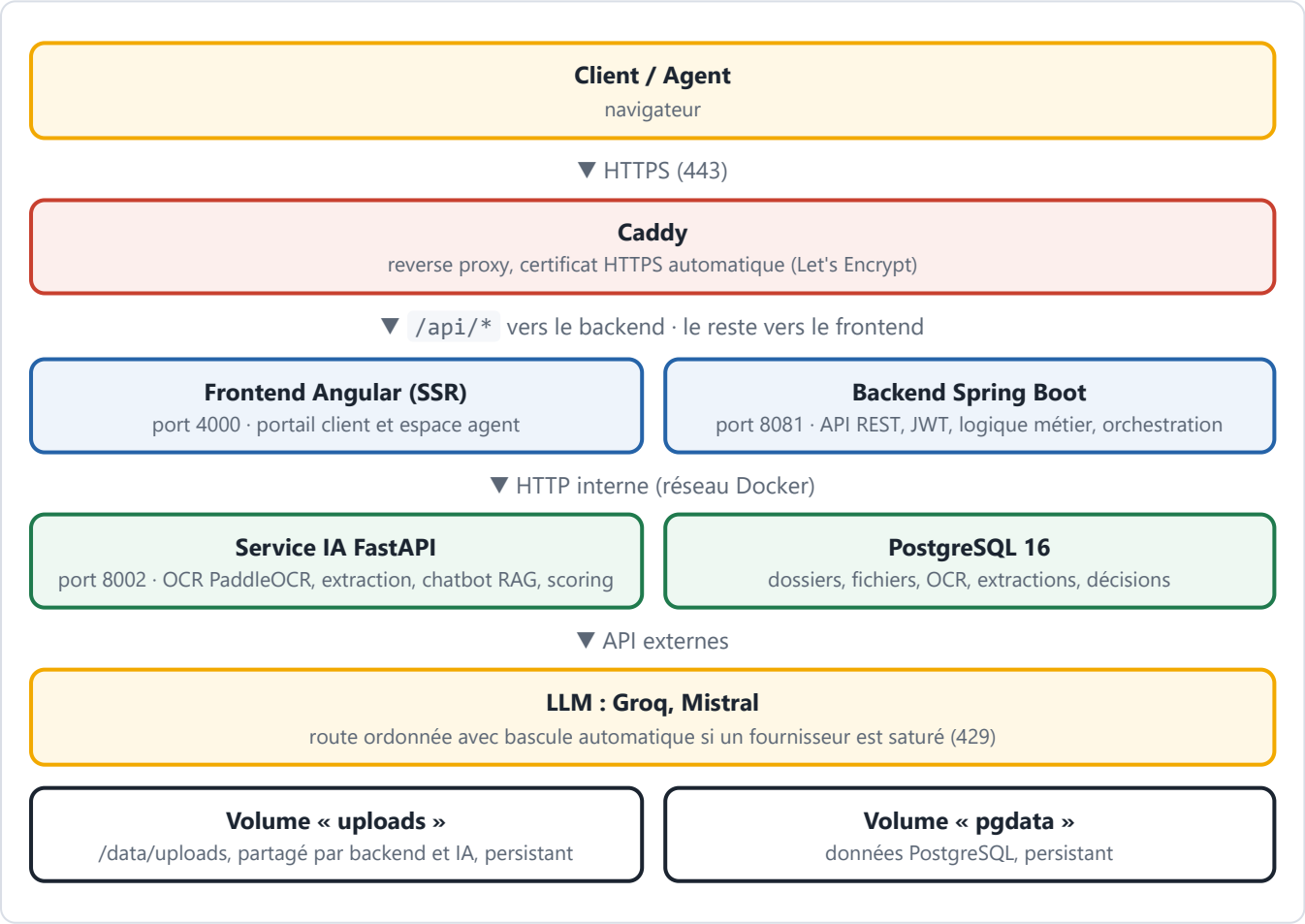
Le projet tourne en production sur une machine virtuelle Azure, déployée par Terraform et Docker Compose, derrière un reverse proxy HTTPS.

Ce qui a été fait pendant ce travail

Chantier	Résultat
Audit du projet	Relecture complète : défauts de sécurité, appels LLM en double, texte tronqué, calculs optimistes.
Déploiement Azure	VM créée par Terraform sur un abonnement étudiant (contraintes de région, de quota et de capacité résolues), stack Docker Compose, HTTPS automatique, mise à jour par GitHub Actions.
Image du service IA	Build corrigé : versions de PaddleOCR / PaddlePaddle alignées (3.x), dépendances numériques figées.
OCR	Photos (JPG, PNG, TIFF...) acceptées, format détecté par le contenu du fichier, formats non supportés refusés avec un message clair, fusion arabe + français, raison d'échec affichée à l'agent.
Fichiers du client	Les dépôts du client n'étaient pas dans le volume persistant, donc perdus à chaque redéploiement. Corrigé.
Scoring	Plus de troncature à 4 000 caractères : budget de 12 000 caractères réparti entre les documents. Appel de scoring en double supprimé, attente artificielle de 3 s supprimée.
Chatbot	L'indexation n'appelle plus le LLM pour rien, et chaque document est indexé avec son propre libellé.

Pipeline d'analyse	Deux étapes réellement verrouillées : « Vérifier les documents » puis « Analyser ». Contrôle du CIN avec confirmation de l'agent, et refus côté serveur (409) sans cette confirmation.
Classification des documents	Branchée dans le pipeline : un document déposé dans le mauvais emplacement est détecté (cascade règles, embeddings, LLM). Un désaccord bloque « Analyser » jusqu'à confirmation de l'agent, et l'extraction JSON est adaptée au type réel du document.
Calcul du crédit	Taux d'intérêt obligatoire, dettes inconnues jamais comptées comme zéro, nouvelle décision « À COMPLÉTER » : un dossier incomplet ne sort plus « éligible ».
Écran de résultat	Affiche tout ce que le moteur calcule, plus le cas « à compléter », le score provisoire et le taux utilisé. E-mail client adapté.
Identité visuelle	Icône et titre de l'onglet : l'icône Angular par défaut est remplacée par l'emblème Attijari.
Données de test	Dossier fictif complet de 5 documents pour le CIN 12015060 (versions natives et scannées).

2. Architecture



Rôle de chaque brique

Brique	Rôle	Choix technique
--------	------	-----------------

Frontend	Portail client (inscription, demande, dépôt de pièces, historique) et espace agent (dossiers, pipeline, résultat, chatbot, export).	Angular, composants standalone, rendu serveur (SSR)
Backend	Authentification, gestion des dossiers et fichiers, orchestration du pipeline , calcul de cohérence du CIN, sauvegarde des résultats, e-mails.	Spring Boot, JPA/Hibernate, JWT, Spring Mail
Service IA	OCR, extraction structurée par LLM, détection de dettes par règles, scoring (LLM + règles), chatbot sur le dossier.	FastAPI, PaddleOCR 3.x, PyMuPDF, MiniLM (embeddings), Pydantic
Base de données	Source de vérité : dossiers, fichiers, texte OCR, extractions JSON, analyse agent, décision finale (avec le résultat complet en JSON).	PostgreSQL 16
Proxy	Termine le HTTPS, route <code>/api</code> vers le backend, limite à 100 Mo par requête.	Caddy 2

Déploiement

- **Infrastructure** : Terraform crée le groupe de ressources, le réseau, la VM Linux et un script `cloud-init` qui installe Docker et écrit les fichiers de configuration (`.env` , `ai.env`) dans `/opt/credisense` .
- **Images** : le workflow GitHub Actions construit les trois images (backend, frontend, ai) et les publie sur GHCR, puis met la VM à jour par SSH. Sur la VM actuelle, les images ont été construites localement (`IMAGE_TAG=latest` , `--pull never`).
- **Secrets** : clés LLM, SMTP, JWT et mot de passe base viennent des fichiers d'environnement, jamais du code.
- **Abonnement étudiant** : région limitée par une politique, quota par famille de VM, erreurs de capacité (`SkuNotAvailable`). Le script `recover-state.sh` et les commentaires de `terraform.tfvars.example` documentent ces cas.

3. Le pipeline de A à Z

Le pipeline a deux acteurs : le client dépose, l'agent vérifie puis analyse. Chaque étape a un point d'entrée précis dans le code.

A

Le client crée sa demande

Le portail client (`/client/demande`) envoie le montant, la durée et le type de contrat à `POST /api/public/demande` . Un dossier est créé en base avec le statut `EN_ATTENTE` .

B

Le client dépose ses pièces

`POST /api/public/upload` (`DemandeCreditController`). Le nom du fichier et le CIN sont assainis, le fichier est écrit dans `upload.base-path` (volume persistant `/data/uploads`), puis une ligne `Fichier` est créée avec le type de document choisi par le client (« AUTRE » par défaut).

C

L'agent ouvre le dossier

L'espace agent charge les fichiers (`GET /api/dossiers/{id}/fichiers`). Chaque document porte son statut OCR (`ocrStatut` , `ocrErreur`), son JSON extrait et sa cohérence de CIN. Le composant `pipeline-analyse` affiche quatre étapes : documents, cohérence du CIN, analyse, score.

D**Étape 1 : « Vérifier les documents »**

`POST /api/fichiers/verifier-dossier/{id} → verifierDossier() → lireEtExtraire()`.

1. **Saut des documents déjà traités** : un document est « déjà vérifié » s'il a un texte OCR réussi et une extraction non vide. Il n'est ni relu ni renvoyé au LLM.
2. **OCR** : le backend envoie le fichier au service IA (`POST /ocr`). Le format est détecté par les premiers octets du fichier (PDF, JPG, PNG, TIFF, BMP, WebP). Un PDF natif est lu directement avec PyMuPDF. Un scan ou une photo passe par PaddleOCR : détection des zones, lecture par le modèle arabe et par le modèle latin, fusion zone par zone, regroupement par ligne, normalisation Unicode. Le Word et le HEIC sont refusés avec un message explicite.
3. **Enregistrement** : `ocr_results` reçoit le texte, le statut (`SUCCESS` ou `FAILED`) et la raison de l'échec.
4. **Classification** : le texte est envoyé à `POST /ai/classify-hybrid` avec le type déclaré par le client. Le service renvoie le type détecté, sa confiance, la méthode, et un bloc « contrôle » : concordant, en désaccord, ou sans conclusion (section 4). Le résultat est enregistré sur le fichier (`typeDetecte` , `confianceType` , `methodeType` , `typeConflit`). Si la classification tombe en panne, le pipeline continue avec le type déclaré.
5. **Extraction JSON adaptée au type** : le texte est envoyé à `POST /ai/extract-json` avec le **type retenu** (le type détecté s'il est fiable, sinon le type déclaré). Le schéma demandé au LLM et les consignes dépendent de ce type : une CIN ne demande que nom, prénom et numéro ; une fiche de paie demande le « net à payer » et pas le brut ; un relevé demande les échéances de crédit et les incidents. Le résultat est validé par Pydantic, avec un score de confiance selon le type. Une extraction vide ou en échec n'est **pas** enregistrée : le document ne peut donc pas être « validé » à tort.
6. **Cohérence du CIN** : `toResponse()` compare, en chiffres uniquement, le CIN extrait de chaque document au CIN du client. Résultat : cohérent, incohérent ou sans objet (justificatif de domicile par exemple).

E**Verrou : le bouton « Analyser »**

Il n'est actif que si les étapes 1 et 2 sont valides. Si seul le CIN est incohérent, ou si le contenu d'un document ne correspond pas à son type déclaré, l'agent peut continuer après une **confirmation explicite** (il a vérifié à la main). Le serveur applique la même règle : `POST /api/fichiers/analyser-dossier/{id}` répond **409** (`CIN_INCOHERENT` ou `TYPE_INCOHERENT`) sans le paramètre `confirmerIncoherence=true` . Une seule confirmation couvre les deux cas. Aucun appel LLM n'est gaspillé sur un dossier douteux.

Après la confirmation, le pipeline passe au vert. Le contrôle manuel de l'agent vaut validation : les étapes « Vérification CIN » et « Validation des documents » deviennent vertes avec la mention « incohérence confirmée manuellement par l'agent ». C'est vrai aussi à la réouverture d'un dossier déjà analysé, puisque le serveur refuse d'analyser un dossier incohérent sans confirmation. La confirmation ne masque rien d'autre : un nom manquant sur la CIN ou un document requis absent restent en erreur.

F

Étape 2 : « Analyser » (`analyserEtScorer()`)

1. Relit uniquement ce qui manque (normalement rien).
2. Indexe les textes pour le chatbot (`POST /ai/chat/index`), sans appel LLM.
3. Assemble le texte du dossier avec un en-tête par document (`=== TYPE (nom) ===`), avec le type détecté quand il contredit le type déclaré) et ajoute en tête la **demande du client** (montant, durée, contrat), absente de tous les documents.
4. Vérifie l'identité : le nom et le prénom du client dans le dossier sont comparés à ceux lus dans les documents (`alerteIdentite`). Cette vérification relance une extraction LLM sur le texte complet du dossier.
5. Appelle le scoring : `POST /ai/score/consommation` .

G

Le scoring dans le service IA (`AgentService.analyser_consommation`)

1. **Cache** : la clé contient le texte, la version des règles, le taux et la taille maximale. Changer l'un d'eux ne ressort jamais une ancienne analyse.
2. **Budget de texte** : 12 000 caractères répartis équitablement entre les documents. Un document trop long garde son début et sa fin. Aucun document n'est supprimé. Un avertissement signale tout raccourcissement.
3. **Un seul appel LLM** (tâche « analyse », température 0,1, JSON imposé) : il lit le dossier et propose revenus, contrat, ancienneté, dettes, points forts, risques, plan, score et décision. Le JSON tronqué est réparé automatiquement.
4. **Dettes** : détecteur par règles dans le relevé bancaire (`dettes_releve.py`) et valeur lue par le LLM. On garde la plus élevée, avec un avertissement en cas d'écart. Un « 0 » sans relevé lisible n'est pas une preuve : la dette reste inconnue.
5. **Couche déterministe** (`appliquer_regles`) : le LLM n'a plus la main sur les chiffres. Mensualité, taux d'endettement (DTI), capacité d'emprunt et simulations sont recalculés en Python (section 4).
6. **Garde-fous** : un critère bloquant en échec ramène « ELIGIBLE » à « CONDITIONNEL » (score plafonné à 59). Des données indispensables manquantes transforment toute décision favorable en « À COMPLÉTER ».

H

Sauvegarde et statut

Le backend enregistre : `AgentAnalysis` (score, revenus, endettement, décision), `DecisionFinale` (justification, message au client, **résultat complet en JSON**) et met à jour le statut du dossier : `ELIGIBLE` → `APPROUVE` , `REFUS` → `REFUSE` , toute autre décision → `EN_COURS` .

I

Résultat pour l'agent et le client

L'écran de résultat (`credit-result`) affiche la décision, le score (marqué « provisoire » si des données manquent), la liste des données manquantes, les contrôles réglementaires, la capacité, les simulations par durée, les points forts, les points de vigilance, les conditions et le taux utilisé. `POST /api/dossiers/{id}/send-result-email` envoie au client un message adapté à la décision. Pour « À COMPLÉTER », il ne cite que des pièces que le client peut fournir, jamais un réglage technique.

Le chatbot (`POST /api/chatbot/question` → `/ai/chat`) répond aux questions de l'agent sur le dossier par recherche dans les documents indexés (RAG), avec masquage des RIB, IBAN et CIN.



4. Le moteur de décision

Qui fait quoi : le LLM et les règles

L'« agent IA » est **un appel LLM suivi de règles déterministes**. Ce n'est pas un agent autonome avec des outils. Le LLM *lit* les documents. Les règles *calculent et contrôlent*. Ce partage est volontaire : un chiffre financier doit être reproductible et vérifiable.

Formules

- **Mensualité** : annuité constante, $M = C \times i / (1 - (1 + i)^{-n})$, avec $i = \text{taux annuel} / 12$.
- **Taux d'endettement (DTI)** : $(\text{mensualité} + \text{dettes mensuelles}) / \text{revenu net} \times 100$.
- **Mensualité maximale** : $30 \% \times \text{revenu} - \text{dettes}$. Le **montant finançable** en découle (inverse de la formule d'annuité), plafonné à $5 \times$ le salaire.
- **Simulations** : durées 12, 24, 36, 48, 60 et 84 mois, plus la durée demandée. Pour chacune : mensualité, DTI, coût total, statut.

Les contrôles

Contrôle	Seuil	Bloquant	Si la donnée manque
Taux d'intérêt appliqué	taux configuré, entre 0 et 50 %	oui si absent	« non configuré » : aucun calcul
Taux d'endettement	< 30 % OK · 30–35 % attention · > 35 % KO	oui	« inconnu », aucune valeur inventée
Dettes existantes	relevé bancaire lisible	oui si inconnues	« à confirmer »
Plafond du montant	$\leq 5 \times$ salaire net	oui	à vérifier
Durée	≤ 84 mois	oui	à vérifier
Ancienneté dans l'emploi	≥ 6 mois	oui	à vérifier (attestation)
Type de contrat	CDI / fonctionnaire favorable	non	à vérifier
Incidents de paiement	0	si > 0	à vérifier (centrale des risques)
Âge en fin de crédit	≤ 70 ans	oui	contrôle non affiché

À savoir. Les seuils (30 %, 35 %, $5 \times$ salaire, 84 mois, 6 mois, 70 ans) sont attribués à la réglementation BCT dans le code, mais je ne les ai pas vérifiés auprès d'une source officielle. À valider avec la banque avant tout usage réel.

Les décisions possibles

Décision	Sens	Statut du dossier
----------	------	-------------------

ELIGIBLE	Tout est connu, aucun critère bloquant en échec.	APPROUVE
CONDITIONNEL	Acceptable avec garanties, ou un critère bloquant est en échec.	EN_COURS
REFUS	Refus fondé sur un critère connu.	REFUSE
A_COMPLETER (nouveau)	Des informations indispensables manquent : aucune décision n'est rendue, la liste des manques est affichée, le score est « provisoire » et plafonné à 59.	EN_COURS
INDETERMINE	Le LLM a échoué ou est saturé.	EN_COURS

La classification des documents

Problème. Le type d'un document vient de l'emplacement choisi par le client à l'upload. Un relevé bancaire déposé dans « fiche de paie » serait traité comme une fiche de paie : revenu et dettes faux. La classification vérifie la *nature réelle* du document.

Cascade à trois niveaux (DocumentClassifierService) :

Niveau	Fonctionnement	Coût
0. Règles	Mots-clés par type, en trois forces : titres (« ATTESTATION DE TRAVAIL », « BULLETIN DE PAIE »...), forts et faibles . Le poids d'un titre dépend de sa position : plein poids tout en haut du texte, poids réduit plus bas. Une attestation qui cite « la carte d'identité nationale » n'est donc pas classée CIN. Texte normalisé (sans accents ni voyelles arabes), français et arabe. Les règles répondent seulement si le verdict est net (score ≥ 5 et écart ≥ 3), sinon elles s'abstiennent, et ne répondent jamais « AUTRE ».	instantané, gratuit, reproductible
1. Embeddings	Similarité sémantique (MiniLM multilingue) avec des phrases de référence. Si la similarité dépasse 0,55, verdict retenu.	quelques ms, local
2. LLM	Seulement dans la zone grise (similarité entre 0,20 et 0,55). Si le LLM est indisponible, on garde le verdict des embeddings.	un appel LLM, rare

Contrôle déclaré / détecté (controle_type.py). On ne signale un conflit que si le verdict est **fiable**. Chaque méthode a son seuil : règles 0,80, embeddings 0,55, LLM 0,70 (leurs échelles de confiance ne sont pas comparables). Un verdict incertain, « AUTRE » ou un type non déclaré ne produit jamais de conflit : il ne bloque rien. En cas de conflit fiable, le **type détecté** pilote l'extraction JSON et le texte vu par l'IA de scoring ; le type déclaré n'est pas écrasé.

Choix de conception. Un désaccord bloque « Analyser » jusqu'à confirmation de l'agent, comme pour le CIN : la banque garde la main, et aucune correction automatique n'est faite.

Traçabilité : le popup « Détail de la classification »

L'agent doit pouvoir voir *exactement* ce qui s'est passé. Le service IA renvoie donc, avec chaque verdict, la **trace de la cascade** niveau par niveau : consulté ou non, a tranché ou non, et pourquoi. Le backend la conserve sur le document (colonne classificationJson) et l'écran l'affiche.

- **Sur chaque document** : une ligne lisible sans rien ouvrir, par exemple « Détecté : Relevé bancaire · règles par mots-clés · confiance 90 % ». Une similarité d'embeddings est affichée comme telle (« similarité 0,56 »), jamais comme un pourcentage.

- **Popup** (bouton « Détail de la classification ») : type déclaré, type détecté, type utilisé pour l'extraction, verdict (conforme, contradiction, sans conclusion, type complété), puis les trois niveaux :
 - *Règles* : mots-clés reconnus, classement des candidats avec leurs scores, seuils à atteindre.
 - *Embeddings* : meilleur candidat, similarité, seuil, candidats.
 - *LLM* : réponse, confiance, justification, fournisseur ; ou « indisponible » avec la raison.Un niveau non consulté est indiqué comme tel, avec la raison (« les règles ont tranché »). Le popup montre aussi le CIN lu dans le document face au CIN du client.

Mesures

Jeu	Résultat
Embeddings seuls, 5 documents natifs du dossier de test	3 / 5 : l'attestation de travail est lue comme CIN (0,407 contre 0,403 pour la bonne réponse) et le relevé bancaire aussi. Seule la CIN dépasse le seuil de 0,55 : les 4 autres dépendraient du LLM.
Règles, 5 natifs + 5 scans (texte OCR réel)	10 / 10 , aucune abstention.
Règles, vraie attestation de travail (document qui n'est pas dans le dossier de test)	correct . Une première version s'abstenait : la mention de la CIN en haut de page faisait presque jeu égal. C'est ce cas qui a conduit au poids selon la position.
Règles, attestation de réussite d'une école (OCR réel)	abstention : elle n'est pas classée « attestation de travail ». Le LLM tranche ensuite.

À lire avec prudence. Le dossier de test a été généré par nous : ses titres ressemblent à nos mots-clés, donc le 10 / 10 est optimiste. Les deux documents réels (attestation de travail, attestation de réussite) sont un échantillon trop petit pour donner un pourcentage. La précision réelle ne sera connue qu'avec de vrais documents de la banque (CIN, fiches de paie, relevés d'Attijari).

Le chatbot RAG : indexation, découpage, recherche

RAG (Retrieval-Augmented Generation) : on ne donne pas tout le dossier au LLM à chaque question. On *cherche* d'abord les passages utiles, puis on les lui donne avec la question. Fichier principal : `chatbot_service.py` .

Indexation = préparer les documents pour qu'on puisse les chercher vite, comme l'index d'un livre. Elle se fait une fois par analyse : (1) découper les textes OCR en chunks, (2) transformer chaque chunk en vecteur de 384 nombres avec le modèle d'embedding (MiniLM multilingue), (3) ranger les vecteurs dans **FAISS** (`IndexFlatIP` , recherche exacte par cosinus), plus un index lexical **BM25**. Les index sont **en mémoire**, un par dossier ; les textes viennent de PostgreSQL (`ocr_results`).

Découpage structurel et sémantique (`decoupage_rag.py`)

Type de texte	Découpage
Relevé bancaire	Un chunk par opération (une ligne datée). Chaque chunk répète l'en-tête « Date Libellé Débit Crédit Solde », et le sens crédit / débit est déduit du solde (ancien solde + montant = nouveau solde : crédit). Si le solde ne colle pas, aucun sens n'est inventé.

Document court (CIN, attestation...)	Un seul chunk : il n'est jamais coupé.
Texte libre	Coupure aux changements de sujet (semantic chunking) : pour chaque frontière, on compare la moyenne des embeddings des 2 lignes avant à celle des 2 lignes après, et on coupe aux frontières dont la similarité est parmi les plus basses.
Tous	Taille maximale de 450 caractères, ce que le modèle lit réellement (128 tokens). Une coupure forcée par la taille garde une ligne de chevauchement ; une coupure sémantique n'en a pas besoin.

Avant, le découpage était positionnel (2 lignes avant, 3 après) : sur un relevé, une opération était coupée et les montants perdaient leur en-tête de colonnes.

Recherche hybride (`recherche_hybride.py`)

- **Sens** : FAISS compare le vecteur de la question à ceux des chunks. « salaire » retrouve « NET A PAYER », en français comme en arabe.
- **Mots exacts** : BM25 (écrit en Python pur, sans dépendance) retrouve les montants, noms et numéros que les embeddings pèsent mal. Un montant « 2 100,000 » donne les jetons « 2100 » et « 2100000 ».
- **Fusion RRF** (Reciprocal Rank Fusion) : un chunk bien classé par l'une OU l'autre remonte.
- **Couverture** : au moins 2 chunks de chaque document, puis complément jusqu'à 12 chunks.

Mémoire de conversation

Problème. Chaque question était traitée seule. À « Et depuis quand ? », le modèle ne savait pas de quoi on parlait, et surtout **la recherche non plus** : une relance ne contient aucun mot utile pour trouver les bons passages.

1. **Envoi de l'historique** : Angular garde déjà les messages et envoie maintenant les 6 derniers échanges réels avec chaque question (sans la salutation ni les messages d'erreur, qui n'ont pas été dits au modèle). Le contrôleur Java les transmet à `/ai/chat`.
2. **Historique donné au LLM** : les tours précédents sont insérés au format chat (system, user, assistant, user), tronqués à 6 messages de 600 caractères. Le prompt système précise que l'historique sert à comprendre la question de suivi et n'est pas une source de vérité : une valeur citée plus tôt n'est une donnée du dossier que si elle figure aussi dans le contexte.
3. **Question de recherche complétée** : une question « relance » (courte, commençant par « et », « mais », « pourquoi »..., ou contenant « précédent », « même », « cela »...) est cherchée avec la ou les questions précédentes de l'agent. C'est une règle, sans appel LLM de plus.
4. **Sécurité de l'historique** : il vient du navigateur, donc il n'est pas cru. Seuls les rôles « user » et « assistant » sont gardés (un faux message « system » est ignoré), les CIN, RIB et IBAN sont masqués, les messages sont tronqués. Le nettoyage est fait côté Java et refait côté Python, qui fait foi.

Résultat mesuré (vrai modèle MiniLM, après « Y a-t-il une échéance de prêt prélevée ? ») : la relance « Et depuis quand ? » ramenait sans mémoire l'attestation d'emploi (hors sujet) ; avec mémoire elle retrouve les trois prélèvements du prêt dans le relevé.

Génération et garde-fous

Les 12 chunks sont envoyés au LLM étiquetés « [Source 3 : Fiche de paie] ». CIN, RIB et IBAN sont masqués avant l'envoi et dans la réponse. Le prompt impose de ne répondre que sur le dossier, de traiter toute

donnée absente du contexte comme une hypothèse, et de lire correctement les montants tunisiens (« 4 800,000 » vaut 4800 DT).

Mesures sur le dossier de test (vrai modèle MiniLM)

Constat	Résultat
Relevé de 155 lignes	Avant : 39 chunks de 6 lignes qui coupaient les opérations. Maintenant : 32 chunks, une opération chacun, avec l'en-tête des colonnes et le sens déduit.
Question « Que signifie 250,000 ? »	Similarité sémantique seule : 0,27 (faible). BM25 : 5,60. Les 3 premiers résultats sont bien les opérations de 250,000 du relevé.
Question « montant du virement de salaire »	Les opérations de salaire du relevé arrivent aux rangs 4 et 7 sur 12 ; les 5 documents du dossier sont couverts.
Taille maximale d'un chunk	430 caractères (limite : 450).

Limites. Index en mémoire : perdu au redémarrage du service IA (il se reconstruit à la prochaine analyse, en attendant le chatbot répond « pas encore de données »). La conversation n'est pas conservée en base : elle vit dans le navigateur et disparaît quand on quitte la page (une table de messages par dossier la rendrait consultable et traçable). La détection des relances est une règle de mots, pas une compréhension : une question longue mais dépendante de la précédente n'est pas complétée. Pas de re-classement après la recherche. Le sens crédit / débit est une déduction, signalée comme telle dans le chunk. Ces mesures portent sur nos documents de test : la qualité réelle sera validée avec de vrais relevés.

Le détecteur de dettes (`dettes_releve.py`)

Il lit le texte du relevé, regroupe les opérations par date (un PDF natif met chaque cellule du tableau sur une ligne séparée), repère les mots d'échéance de prêt sur du texte sans accents, exclut salaire, total et solde, et retient le plus fort total mensuel des deux derniers mois. Une échéance déjà soldée n'est plus comptée.

Limite assumée. C'est une heuristique sur du texte. Le résultat est toujours présenté comme « détecté, à confirmer ».

5. Détail des changements

5.1 Déploiement et infrastructure

- Stack de production : `deploy/docker-compose.prod.yml` (postgres, ai, backend, frontend, caddy) et `deploy/Caddyfile`.
- Terraform (`infra/terraform`) : VM, réseau, `cloud-init`. Variables `location` et `vm_size` à fixer ensemble, sinon une relance tente de recréer la VM.
- Configuration par variables d'environnement (`application.properties`) : base, JWT, URL du service IA, chemin des uploads, SMTP, CORS.
- `deploy/redeploy.sh` : script de mise à jour de la VM (non commité à ce stade).

5.2 Service IA

- **Image** : PaddleOCR 3.4.0 et PaddlePaddle 3.3.1, modèles préchargés, scipy/skimage/sklearn figés, zlib système forcé. La cause réelle de l'échec était le décalage de versions Paddle.
- **OCR** (`ocr_service.py`) : détection du format par octets, images réduites au-delà de 4 000 px, fusion arabe/latin, limite de 25 Mo, fichier temporaire toujours supprimé.
- **Classification** : `classification_regles.py` (règles par mots-clés, niveau 0 de la cascade), `controle_type.py` (comparaison déclaré / détecté, seuils par méthode), endpoint `/ai/classify-hybrid` enrichi du bloc « controle ».
- **Extraction JSON** (`groq_service.py`) : schéma et consignes par type de document ; le type classé arrive jusqu'au prompt.
- **Texte du dossier** (`dossier_texte.py`) : budget par document.
- **Scoring** (`agent_service.py`) : règles, taux obligatoire, données manquantes, cache versionné.
- **Chatbot** : index construit sans LLM, un libellé par document, journaux sans données personnelles ; découpage structurel et sémantique (`decoupage_rag.py`) ; recherche hybride sens + mots exacts avec fusion RRF (`recherche_hybride.py`) ; mémoire de conversation (historique transmis au LLM, relances complétées pour la recherche).

5.3 Backend

- `FichierServiceImpl` : séparation vérifier / analyser, saut des documents déjà lus, une ligne OCR par fichier (mise à jour au lieu d'un doublon), extraction en échec non enregistrée, garde 409 sur le CIN, message clair si le fichier a disparu du disque.
- `Fichier` : nouveaux champs `typeDetecte`, `confianceType`, `methodeType`, `typeConflit`, `classificationJson` (réponse complète de la classification, trace comprise ; colonnes créées automatiquement au démarrage par `ddl-auto=update`).
- `FichierController` : endpoints `verifier-dossier` et `analyser-dossier`, avec les codes d'erreur `CIN_INCOHERENT` et `TYPE_INCOHERENT`.
- `DemandeCreditController` : écriture dans le volume persistant, CIN et nom de fichier assainis.
- `DossierController` : e-mail de résultat adapté à « À COMPLÉTER », libellés lisibles, bloc de score omis tant que la décision n'est pas rendue.
- `ChatbotController` / `ChatbotService` : la requête accepte un historique facultatif, nettoyé (6 derniers messages, rôles connus, 600 caractères) puis transmis au service IA.

5.4 Frontend

- `upload-section` : deux boutons réellement verrouillés, confirmation du CIN, affichage de la raison d'échec OCR.
- `pipeline-analyse` : calcule si l'analyse est autorisée, directement ou après confirmation (CIN incohérent ou type contredit).
- `upload-section` : message « déposé comme X, le contenu ressemble à Y » sur le document, confirmation unique pour les deux types d'incohérence, ligne de résumé de la classification sur chaque document et popup de détail (cascade niveau par niveau, CIN lu face au CIN du client).
- `pipeline-analyse` : une incohérence confirmée par l'agent (ou déjà analysée) passe les étapes 1 et 2 au vert, avec mention.
- `credit-result` : écran de résultat complet (section 3-l). Le modèle exporte `mapperResultat`, qui recopie tous les champs. Avant, plusieurs champs étaient perdus à la copie, donc invisibles.

- chat-assistant : envoie les derniers échanges réels avec chaque question.
- Icône et titre de l'onglet.

6. Tests

Résultats

Couche	Résultat	Ce qui est couvert
Service IA (Python)	142 / 142 passent	test_agent_regles (18) : taux obligatoire (absent, illisible, en pourcentage), dette inconnue jamais traitée comme zéro, DTI, capacité, simulations, décision « À COMPLÉTER », cohérence LLM/règles, clé de cache. test_dettes_releve (12) : relevé natif et scanné, prêt soldé, salaire exclu, relevé illisible, absence de relevé. test_dossier_texte (9) : répartition du budget, début et fin conservés, aucun document perdu. test_chatbot_indexation (5) : un libellé par document, aucun appel LLM. test_classification_regles (24) : documents français et arabe, OCR sans accents, attestation qui cite la CIN, relevé qui mentionne une facture, attestation de réussite (abstention), cascade (les règles court-circuitent embeddings et LLM) ; diagnostic expliqué (verdict net, abstention, texte court, aucun mot-clé) ; trace de la cascade dans chaque cas (règles seules, trois niveaux, embeddings sûrs, similarité trop basse, LLM en panne, sérialisable en JSON). test_controle_type (10) : conflit seulement si le verdict est fiable, seuil par méthode, type non déclaré, entrées mal formées. test_extraction_par_type (10) : schéma par type, identité toujours demandée, le type classé change le prompt, cache distinct selon le type. test_decoupage_rag (19) : un chunk par opération, en-tête de colonnes répété, sens crédit / débit déduit du solde (et non inventé si le solde ne colle pas), document court non coupé, aucun chunk au-delà de 450 caractères, aucune ligne perdue, coupure au changement de sujet, embeddings en panne sans effet. test_recherche_hybride (18) : jetons des montants, BM25 sur montant exact, fusion RRF, retriever hybride (montant exact en tête, couverture des documents, repli sans BM25). test_chatbot_memoire (17) : historique inséré dans l'ordre entre le système et la question, 6 derniers messages, troncature, faux message « system » ignoré, CIN et RIB masqués, entrées invalides, détection des relances, chaîne de relances, la recherche suit le sujet de la conversation.

Backend (Java)	32 / 32 passent	FichierServiceImplTest (17) : garde 409 sur le CIN, retraitement évité, extraction en échec non enregistrée ; nouveaux (9) : conflit de type enregistré et extraction faite avec le type détecté, verdict incertain sans conflit, type non déclaré complété, panne de classification sans blocage, verrou « type contredit », CIN et type signalés ensemble, confirmation, le scoring voit le vrai type. DossierControllerEmailTest (3) : e-mail pour « À COMPLÉTER », sans réglage technique, sans bloc de score. DossierControllerFichiersTest (3) : le détail de la classification et sa trace sont transmis à l'écran ; document non classé : valeur nulle ; détail illisible : la liste n'échoue pas. ChatbotServiceMemoireTest (9) : historique transmis dans l'ordre, absent ou null donne une liste vide, 6 derniers messages, troncature, rôles inconnus et messages vides écartés, service IA indisponible, contrôleur avec et sans historique.
Frontend (Angular)	57 / 57 passent sur nos fichiers (67 sur 73 au total)	pipeline-analyse (23) : règles de verrouillage, dont 6 sur le conflit de type et 8 sur la confirmation de l'agent (étapes 1 et 2 au vert avec mention, relance sans redemander, dossier déjà analysé, CIN et type ensemble, identité manquante et document requis absent non masqués). upload-section (17) : verdicts, ligne de résumé (pourcentage pour les règles, similarité pour les embeddings), cascade (règles seules, trois niveaux, LLM en panne), ouverture, fermeture et contenu du popup, CIN lu face au CIN du client, confirmation de l'agent et remise à zéro au changement de dossier. chat-assistant (7) : la salutation et les messages d'erreur ne sont pas envoyés comme historique, les deux premiers échanges dans l'ordre, 6 derniers messages, la question en cours jamais dans l'historique. credit-result (4) : affichage complet, cas « à compléter », absence de blocs parasites. credit-analysis-result.model (6) : aucun champ perdu, valeurs par défaut sûres, score 0 conservé.
Build	OK	ng build passe. Avertissements sur le budget CSS et NG8107, déjà présents.

Vérification sur le dossier de test (CIN 12015060)

- Taux fixé à 10 % pour le test : dette détectée de **250 DT**, mensualité de **228,263 DT**, taux d'endettement de **22,77 %**.
- Même dossier **sans taux** : décision « À COMPLÉTER », aucun chiffre inventé.

Échecs connus, non liés à nos changements.

- 6 tests frontend** générés par défaut par Angular échouent : `app.spec`, `reset-password (×2)`, `verify-email`, `register`, `document.spec` (le test du chat, lui aussi généré par défaut, a été remplacé par de vrais tests). Aucun de ces fichiers n'a été modifié. Il faut soit les écrire pour de bon, soit les retirer.
- DemoApplicationTests (Java)** échoue : il a besoin d'une vraie base de données.

Ce qui n'est pas testé

- Aucun essai dans un vrai navigateur après les derniers changements de l'écran de résultat.
- Aucun essai avec un vrai appel LLM sur la nouvelle logique de décision : les tests simulent la réponse du LLM.
- Les seuils réglementaires ne sont pas validés par une source officielle.
- Aucune mesure de qualité du **score** : il vient toujours du LLM. La phase 2 prévoit une grille de score déterministe.

7. Limites connues et suite

Fonctionnement

Sujet	État
Classification des documents	Branchée. Précision réelle inconnue : les mesures portent sur nos documents de test (voir section 4). Le seuil des embeddings (0,55) reste bas : une similarité n'est pas une probabilité. Non branché : <code>/ai/verify-documents</code> (pièces requises selon l'âge, le contrat, la nationalité) : le portail impose déjà les emplacements, mais ce contrôle ne s'applique pas encore côté agent. La classification n'est lancée qu'à l'étape « Vérifier » : un document lu avant cette version n'a pas de type détecté tant qu'il n'est pas relu.
Score	Produit par le LLM, donc peu reproductible. Prévu : grille de score déterministe avec détail par critère.
Statut automatique	« ELIGIBLE » met le dossier en « APPROUVE » sans validation humaine. À discuter : une banque voudrait probablement une validation de l'agent.
Appels LLM	Une analyse complète fait environ une extraction par document, une extraction supplémentaire pour la vérification d'identité et un appel de scoring. La vérification d'identité pourrait réutiliser les extractions déjà faites.
Vitesse de l'OCR	Environ 42 s par page scannée sur CPU. Pistes : résolution plus basse, taille de détection limitée, oneDNN si la version le permet.
Taux d'intérêt	Doit être renseigné par la banque dans <code>CREDIT_TAUX_ANNUEL</code> . Le 10 % utilisé pour les essais n'est pas un taux réel.

Détail de la classification : à savoir. Il n'existe que pour les documents classés avec cette version : un document lu avant n'a pas de détail (le popup l'indique et propose de relancer « Vérifier les documents », qui ne retraite toutefois pas un document déjà lu et extrait : il faut le renvoyer). L'aspect visuel du popup n'a pas été vérifié dans un navigateur : seule sa structure est testée.

Sécurité (à traiter avant tout usage réel)

- `/api/fichiers/**` , `/api/chatbot/**` , `/api/clients/**` et l'envoi d'e-mail de résultat sont ouverts sans authentification (`permitAll`).
- `/api/fichiers/view/**` sert des fichiers à partir d'un chemin : risque de lecture de fichiers hors du dossier prévu (path traversal).

- Le dossier `Backend/uploads/` contient des pièces d'identité suivies par git : données personnelles à retirer de l'historique.
- Des clés d'API ont été exposées pendant le travail : à **renouveler**.
- Le résultat d'identité (`alerteIdentite`) n'est pas conservé dans le résultat stocké.

Phases suivantes (proposées, pas commencées)

1. **Phase 2** : grille de score déterministe avec détail par critère.
2. **Phase 3** : schémas et graphiques, plans distincts pour le client et pour l'agent, aperçu avant envoi de l'e-mail.
3. **Phase 4** (reste) : jeu d'évaluation avec de vrais documents de la banque pour mesurer la précision de la cascade, recalibrage des seuils, contrôle des pièces requises côté agent.

Mise en ligne de cette version sur la VM

1. Dans `/opt/credisense/ai.env`, ajouter `CREDIT_TAUX_ANNUEL=<taux annuel de la banque en décimal, par exemple 0.10>`. Sans lui, tous les dossiers restent « À COMPLÉTER ».
2. Reconstruire les images ai, backend et frontend (ou laisser GitHub Actions les construire).
3. Vérifier `IMAGE_TAG=latest`, puis `docker compose up -d --force-recreate`.
4. Rafraîchir le navigateur avec Ctrl+F5 pour voir la nouvelle icône.

8. Speech : présenter l'architecture (environ 4 minutes)

Ouverture. Bonjour. Je vais vous présenter CrediSense AI, une application qui aide un conseiller bancaire à étudier une demande de crédit à la consommation. Le problème de départ est simple : un dossier arrive avec cinq ou six documents, souvent scannés, en arabe et en français, et le conseiller doit tout relire pour savoir si le client peut rembourser. Notre application fait cette lecture et ce calcul, puis explique son résultat.

Les trois blocs. L'architecture tient en trois blocs. Devant, une application Angular : un portail où le client dépose sa demande et ses pièces, et un espace où le conseiller travaille. Au milieu, un backend Spring Boot. C'est le chef d'orchestre : il gère les comptes, les dossiers, les fichiers, et il décide de l'ordre des étapes. Derrière, un service d'intelligence artificielle en Python avec FastAPI. Lui, il sait lire les documents, extraire les informations et faire l'analyse. Une base PostgreSQL garde tout, et un reverse proxy, Caddy, protège l'ensemble avec le HTTPS.

Pourquoi séparer le backend et le service IA. C'est un choix volontaire. Le monde de l'IA, avec PaddleOCR et les modèles, vit en Python. Le monde métier, avec la sécurité, les transactions et la base de données, est solide en Java. Chacun reste dans son langage et peut être déployé et mis à jour séparément. Ils se parlent par des appels HTTP simples, dans un réseau privé.

Le parcours d'un dossier. Le client dépose ses pièces. Le conseiller clique sur « Vérifier les documents ». À cet instant, chaque document passe par l'OCR : si le PDF contient déjà du texte, on le lit directement ; si c'est un scan ou une photo, PaddleOCR lit le texte avec deux modèles, un arabe et un français, puis on fusionne les deux lectures. Ensuite un modèle de langage transforme ce texte en données structurées : nom, CIN, salaire, type de contrat, avec des consignes adaptées au type de document. Et le système vérifie que le CIN lu sur chaque document est bien celui du client.

Le verrou. L'analyse ne démarre que si cette vérification est valide. Si le CIN ne correspond pas, le conseiller peut continuer, mais seulement en confirmant explicitement, et le serveur refuse de lui-même sans cette confirmation. Nous ne dépensons donc pas un appel d'IA sur un dossier qui n'est peut-être pas le bon.

Le contrôle des documents. Le client choisit lui-même où il dépose chaque pièce. Rien ne l'empêche de mettre un relevé bancaire à la place de la fiche de paie. Nous vérifions donc la nature réelle de chaque document, avec une classification en trois niveaux. D'abord des règles : le titre du document, « attestation de travail », « bulletin de paie », « relevé de compte », en français comme en arabe. C'est instantané, gratuit et reproductible. Si les règles ne sont pas sûres, on passe à une comparaison sémantique par embeddings, et seulement en dernier recours à un modèle de langage. Nous avons ajouté les règles après une mesure : les embeddings seuls classaient une attestation de travail comme une carte d'identité, parce qu'elle cite le numéro de carte. Si le contenu contredit l'emplacement choisi, l'analyse est bloquée jusqu'à ce que le conseiller confirme, et le type réel sert aussi à adapter l'extraction : on ne demande pas les mêmes champs à une carte d'identité et à un relevé.

Le chatbot. Le conseiller peut aussi poser des questions en langage naturel sur le dossier. Nous utilisons une architecture RAG : les documents sont découpés, transformés en vecteurs par un modèle d'embedding multilingue, et rangés dans un index. À chaque question, on retrouve les passages les plus

proches, par le sens et par les mots exacts, puis un modèle de langage rédige la réponse en citant ses sources. Le découpage tient compte de la structure : un relevé bancaire est coupé par opération, avec ses en-têtes de colonnes, parce qu'un montant n'a de sens qu'avec sa colonne. Le chatbot a aussi une mémoire de conversation : le conseiller peut enchaîner « Et depuis quand ? » sans répéter le sujet, parce que la relance est complétée avec la question précédente avant la recherche.

Le cœur : un agent en deux couches. Pour l'analyse, nous avons fait un choix important. Le modèle de langage lit le dossier et propose une première évaluation. Mais il ne calcule pas les chiffres. Les mensualités, le taux d'endettement, la capacité d'emprunt et les simulations sont calculés par des règles, en Python, avec des formules connues. Ces règles contrôlent aussi les critères réglementaires : endettement sous 30 %, montant plafonné à cinq fois le salaire, durée, ancienneté. Si le modèle dit « éligible » alors qu'un critère bloquant échoue, les règles corrigent. Un chiffre financier doit être reproductible, et on peut l'expliquer.

L'honnêteté du calcul. Dernier point, et c'est celui dont je suis le plus fier : le système ne devine jamais. Si le taux d'intérêt n'est pas configuré, il ne calcule aucune mensualité. Si on ne sait pas si le client a déjà un crédit, il ne suppose pas que non : il détecte les échéances dans le relevé bancaire, et s'il n'y a pas de relevé, la dette reste inconnue. Dans ces cas, la décision n'est ni « éligible » ni « refus », c'est « à compléter », avec la liste précise de ce qu'il manque.

Déploiement. Tout tourne sur une machine virtuelle Azure, créée avec Terraform, donc reproductible. Les cinq services sont des conteneurs Docker orchestrés par Docker Compose. Les fichiers déposés et la base sont sur des volumes persistants, donc ils survivent aux mises à jour. Le code est construit et publié automatiquement par GitHub Actions.

Ce qui reste à faire, et je préfère le dire. Le score affiché vient encore du modèle de langage ; nous préparons une grille de score déterministe. La classification est en place, mais nous ne connaissons pas encore sa précision réelle : nous l'avons mesurée sur nos propres documents de test, ce qui est optimiste. Il nous faut de vrais documents de la banque pour la valider. Côté sécurité, plusieurs routes doivent encore être protégées avant un usage réel. Et le taux d'intérêt doit être fourni par la banque.

Conclusion. En résumé : une architecture à trois couches claire, un pipeline verrouillé étape par étape, un calcul que l'on peut expliquer, et un système qui préfère dire « il me manque une information » plutôt que d'inventer une réponse. Merci, je suis prêt pour vos questions.

Questions probables et réponses courtes

Question	Réponse
Pourquoi un LLM et des règles ?	Le LLM comprend des documents variés. Les règles garantissent des calculs reproductibles et conformes. Chacun fait ce qu'il fait bien.
Est-ce vraiment un « agent » ?	Non au sens autonome. C'est un appel LLM suivi d'une couche de règles. Le pipeline orchestré par le backend joue le rôle de l'agent.
Que se passe-t-il si le LLM est saturé ?	Bascule automatique Groq / Mistral. Si tous échouent, la décision est « indéterminée » avec un message clair, sans bloquer le dossier.
Que se passe-t-il si un document est illisible ?	Le statut OCR « FAILED » et sa raison s'affichent sur le document. Aucune extraction vide n'est enregistrée comme valide.

Comment gérez-vous l'arabe ?	Deux lectures de la même zone, par un modèle arabe et un modèle latin, fusionnées zone par zone.
Les données personnelles sont-elles protégées ?	Le chatbot masque RIB, IBAN et CIN, et les journaux n'affichent plus le texte OCR. Les routes API doivent encore être protégées : c'est notre priorité de sécurité.
Quelle est la précision ?	Calculs : déterministes et testés. Classification : 10 documents sur 10 avec les règles sur notre dossier de test, mais ce dossier est écrit par nous, donc le chiffre est optimiste ; sur les embeddings seuls, 3 sur 5. La vraie précision se mesurera avec des documents réels de la banque. Score : celui du LLM, non mesuré pour l'instant.
Comment fonctionne votre RAG ?	Indexation : découpage structurel et sémantique, embeddings MiniLM multilingue, index FAISS en mémoire plus BM25. Question : recherche hybride (sens plus mots exacts) fusionnée par RRF, au moins 2 passages par document, puis génération par le LLM avec sources citées et identifiants masqués.
Comment gérez-vous la mémoire du chatbot ?	Le navigateur renvoie les 6 derniers messages avec chaque question. Le modèle les reçoit comme historique de conversation, et une question de suivi est complétée avec la question précédente pour la recherche, par une règle simple qui n'ajoute aucun appel LLM. L'historique n'est pas cru : rôles filtrés, identifiants masqués, messages tronqués.
Pourquoi une recherche hybride ?	Les embeddings comprennent le sens mais pèsent mal les valeurs exactes. Sur « 250,000 », la similarité sémantique n'est que de 0,27, alors que BM25 retrouve immédiatement l'opération. Les deux sont complémentaires.
Que se passe-t-il si le client dépose le mauvais document ?	La classification le détecte, le pipeline affiche « déposé comme fiche de paie, le contenu ressemble à relevé bancaire », et « Analyser » reste bloqué jusqu'à ce que le conseiller confirme. Si la classification n'est pas sûre, elle ne bloque rien.